Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251198**
Nama Lengkap	**Moses Merohito**
Minggu ke / Materi	**10 / Tipe Data List**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1 Sifat-sifat List

List dalam Python merupakan struktur data yang menyimpan sekumpulan nilai dengan satu nama. Berbeda dengan string yang terbatas pada urutan karakter, list menawarkan fleksibilitas yang lebih besar karena dapat berisi campuran tipe data seperti karakter, angka (integer dan float), dan bahkan list di dalamnya (list bersarang). Sintaks penulisan list menggunakan tanda kurung siku [] untuk mengapit elemen-elemennya.

```
nilai_ujian = [80,75,70,90,81,84,92,71,65,80,70]
nama_pahlawan = ['Sukarno', 'Diponegoro', 'Jend. Sudirman', 'Cut Nya Dhien']
nilai_campuran = ['Javascript', 20, 34.4, True]
list_dalam_list = [23, [22, 20], 45]
```

Selain perbedaan tipe data yang dapat disimpan, list dan string juga berbeda dalam hal kemudahan perubahan nilai. List di Python bersifat *mutable*, memungkinkan kita untuk mengganti, menambah, atau menghapus elemennya secara langsung pada objek list yang sudah ada. Sementara itu, string bersifat *immutable*. Ini berarti, setiap operasi yang tampak seperti mengubah string sebenarnya akan menghasilkan string baru, dan string aslinya tetap tidak berubah.

```
# definisikan list berisi 4 nilai
data = [10,20,30,40]
# ubah nilai index ke-0 menjadi 50
data[0] = 50
# isinya sekarang: 50, 20, 30, 40
print(data)
```

```
# definisikan sebuah string
nama = 'Antonius Rachmat'
# ubah karakter index ke-0 menjadi Z
nama[0] = 'Z'
# tidak akan mencapai baris ini karena muncul error
# TypeError: 'str' object does not support item assignment
print(nama)
```

Dalam hal penggunaan memori, string dan list berperilaku berbeda. Ketika dua variabel string memiliki nilai yang sama, Python secara efisien akan membuat kedua variabel tersebut mengarah ke satu objek string yang sama di memori. Sebaliknya, meskipun dua list memiliki elemen yang identik, Python akan tetap mengalokasikan ruang memori terpisah untuk masing-masing list, sehingga keduanya menjadi objek yang berbeda.

```
>>> a = 'banana'
>>> b = 'banana'
>>> a is b
True

>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a is b
False
```

MATERI 2 Mengakses dan Mengubah Isi List

Di bawah ini merupakan cara untuk mengakses indeks pada list. List dapat diubah isi dari list dengan menggunakan indeksnya juga. Berikut ini adalah contoh kode untuk menerapkannya:

```
thislist = ["apple", "banana", "cherry"]

print(thislist[1])
print(thislist[-1])
print(thislist[2:5])
print(thislist[:4])

thislist[1] = "jeruk"

print(thislist)

thislist2 = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist2[1:3] = ["blackcurrant", "watermelon"]
print(thislist2)
```

MATERI 3 Fungsi-Fungsi Untuk List

Penambahan list juga bisa lakukan dengan menggunakan operator +, pengulangan menggunakan *. Contoh:

```
thislist = ["apple", "banana", "cherry"]

print(thislist[1])
print(thislist[-1])
print(thislist[2:5])
print(thislist[:4])

thislist[1] = "jeruk"

print(thislist)

thislist2 = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
thislist2[1:3] = ["blackcurrant", "watermelon"]
print(thislist2)
```

Berikut versi parafrase yang lebih ringkas dan jelas:

1. **append()**
   Digunakan untuk menambahkan satu elemen ke akhir list. Elemen yang ditambahkan dianggap sebagai satu objek utuh, meskipun berupa list lain.
2. **extend()**
   Berfungsi menambahkan beberapa elemen ke dalam list. Berbeda dari append(), setiap elemen dari iterable yang ditambahkan akan dimasukkan satu per satu sebagai elemen terpisah.
3. **sort()**
   Digunakan untuk mengurutkan isi list secara langsung (tanpa membuat list baru), biasanya dalam urutan naik atau alfabet.
4. **pop()**
   Digunakan untuk menghapus elemen berdasarkan indeks tertentu, sekaligus mengembalikan nilai elemen yang dihapus tersebut.
5. **del**
   Digunakan untuk menghapus elemen pada indeks tertentu tanpa mengembalikan nilainya.
6. **remove()**
   Berfungsi menghapus elemen dari list berdasarkan nilai yang ditentukan.
7. **reverse()**
   Digunakan untuk membalik urutan elemen dalam list.

Selain itu, Python memiliki fitur **list comprehension**, yaitu cara singkat untuk membuat list baru dari list yang sudah ada. Dengan metode ini, kita bisa melakukan perulangan dan menerapkan

kondisi atau operasi pada setiap elemen, misalnya untuk menghasilkan list boolean yang menunjukkan apakah suatu string mengandung substring tertentu.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if "or" in x]
```

Contoh untuk mengubah semua elemen list menjadi huruf besar.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x.upper() for x in nama]
```

Contoh untuk memeriksa mengambil elemen list yang panjangnya lebih dari 6 huruf.

```
nama = ["anton", "kuncoro", "buntoro", "juworo", "margono"]
newlist = [x for x in nama if len(x) > 6]
```

MATERI 4 List Sebagai Parameter Fungsi

Salah satu karakteristik penting dari list di Python adalah bahwa ketika ia digunakan sebagai parameter fungsi, ia bersifat *mutable*. Akibatnya, setiap perubahan yang Anda lakukan pada list di dalam fungsi akan memiliki efek samping, yaitu mengubah nilai dari list aslinya di luar cakupan fungsi. Ini perlu diperhatikan saat merancang fungsi yang menerima list sebagai input, karena operasi di dalamnya dapat memodifikasi data asli. Contoh:

```
>>> def ubah(data):
...     data[0] = "anton"
...
>>> data = ["a","b","c"]
>>> ubah(data)
>>> data
['anton', 'b', 'c']
```

Keuntungan Menggunakan List sebagai Parameter**:**

1. **Fleksibel**: fungsi dapat menangani berbagai jenis data melalui penggunaan list.
2. **Dapat digunakan ulang**: fungsi bisa dipakai kembali dengan isi list yang berbeda-beda.
3. **Pembagian tugas**: list memudahkan pemecahan pekerjaan menjadi bagian yang lebih kecil dan terstruktur.
4. **Pengelolaan data**: list membantu menata data agar lebih rapi sebelum diproses oleh fungsi.

Contoh lain untuk menghapus elemen pertama pada list:

```
>>> def hapus(data):
...     return data[1:]
...
>>> hapus(data)
['b', 'c']
>>> data
['anton', 'b', 'c']
>>> data = hapus(data)
>>> data
['b', 'c']
```

BAGIAN 2: LATIHAN MANDIRI (60%)

https://github.com/Moses808/71251198_Moses_Merohito

SOAL 1

jupyter 71251198.mosesmerohito Last Checkpoint: 1 minute ago

File Edit View Run Kernel Settings Help

Code JupyterLab

[1]:
```python
def tiga_bilangan_terbaik(lst):
    unique_numbers = set(lst)
    sorted_numbers = sorted(unique_numbers, reverse=True)
    return sorted_numbers[:3]

def main():
    bilangan = []
    print("Masukkan 3 bilangan satu per satu:")

    for i in range(3):
        while True:
            try:
                angka = int(input(f"Input bilangan ke-{i+1}: "))
                bilangan.append(angka)
                break
            except ValueError:
                print("Input harus berupa bilangan bulat valid, coba lagi.

    terbaik = tiga_bilangan_terbaik(bilangan)
    print("3 bilangan terbaik adalah:", terbaik)

if __name__ == "__main__":
    main()
```

```
Masukkan 3 bilangan satu per satu:
Input bilangan ke-1:  22
Input bilangan ke-2:  22
Input bilangan ke-3:  22
3 bilangan terbaik adalah: [22]
```

Penjelasan :

Program tersebut digunakan untuk mengambil 3 bilangan terbesar dari input pengguna, dengan alur sebagai berikut:

Fungsi tiga_bilangan_terbaik(lst) :

- Mengubah list menjadi set untuk menghilangkan duplikasi angka.
- Mengurutkan angka dari terbesar ke terkecil.

- Mengambil 3 nilai teratas.

Fungsi main() :

- Meminta pengguna memasukkan 3 bilangan satu per satu.
- Menggunakan `try-except` untuk memastikan input valid (harus bilangan bulat).
- Menyimpan input ke dalam list.
- Memanggil fungsi untuk mencari 3 bilangan terbaik.
- Menampilkan hasilnya.

Intinya: program ini menerima input angka, memprosesnya (hapus duplikat + urutkan), lalu menampilkan 3 angka terbesar.

SOAL 2

jupyter 71251198.mosesmerohito Last Checkpoint: 7 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python [conda env:base] *

[2]:
```python
def main():
    bilangan = []
    print('Masukkan bilangan satu per satu. Ketik "done" untuk selesai dan lihat rata-rata.')

    while True:
        user_input = input("Masukkan bilangan (atau 'done'): ").strip()

        if user_input.lower() == "done":
            break

        try:
            angka = float(user_input)
            bilangan.append(angka)
        except ValueError:
            print("Input tidak valid, mohon masukkan bilangan atau 'done'.")

    if bilangan:
        rata_rata = sum(bilangan) / len(bilangan)
        print(f"Rata-rata bilangan yang dimasukkan adalah: {rata_rata}")
    else:
        print("Tidak ada bilangan yang dimasukkan.")

if __name__ == "__main__":
    main()
```

```
Masukkan bilangan satu per satu. Ketik "done" untuk selesai dan lihat rata-rata.
Masukkan bilangan (atau 'done'):  21
Masukkan bilangan (atau 'done'):  21
Masukkan bilangan (atau 'done'):  21
Masukkan bilangan (atau 'done'):  21
Masukkan bilangan (atau 'done'):  done
Rata-rata bilangan yang dimasukkan adalah: 21.0
```

Kode tersebut digunakan untuk menghitung rata-rata bilangan yang dimasukkan oleh pengguna secara interaktif.

Penjelasan :

- Program meminta pengguna memasukkan angka satu per satu.
- Input dimasukkan dalam bentuk teks, lalu dicek:
    - Jika pengguna mengetik "done", proses input berhenti.
    - Jika bukan, program mencoba mengubah input menjadi angka (float).
- Jika input valid, angka disimpan ke dalam list bilangan.
- Jika tidak valid, muncul pesan error dan pengguna diminta input ulang.
- Setelah selesai:
    - Jika ada data, program menghitung rata-rata dengan sum(bilangan) / len(bilangan).

- Jika tidak ada data, ditampilkan pesan bahwa tidak ada input.

Intinya**:** program ini mengumpulkan angka secara fleksibel sampai pengguna selesai, lalu menghitung dan menampilkan nilai rata-ratanya.

SOAL 3

jupyter 71251198.mosesmerohito Last Checkpoint: 15 minutes ago

File Edit View Run Kernel Settings Help

Code JupyterLab Python (conda env:base) *

```python
with open("text.txt", "w") as file:
    file.write(teks)

print("File text.txt berhasil dibuat!")
```

```
File text.txt berhasil dibuat!
```

[5]:

```python
def cari_kata_unik(file_path):
    try:
        with open(file_path, 'r') as file:
            artikel = file.read()
        artikel = artikel.lower()
        daftar_kata = artikel.split()
        kata_unik = set(daftar_kata)
        print("Kata-kata unik yang ditemukan:")
        for kata in kata_unik:
            print(kata)

    except FileNotFoundError:
        print(f"File {file_path} tidak ditemukan.")
    except Exception as e:
        print(f"Terjadi kesalahan: {e}")

file_path = input("Masukkan path file teks: ")
cari_kata_unik(file_path)
```

```
Masukkan path file teks:  text.txt
Kata-kata unik yang ditemukan:
energik.
kaki
yang
dan
berolahraga.
jalan,
udara
mereka
beberapa
jakarta.
merasa
bersinar.
matahari
rumah
ringan.
menikmati
membuat
cuaca
makanan
cerah
bahagia
juga
hari
lainnya
sangat
bersepeda,
keluar
sepanjang
untuk
di
taman.
berjalan
segar
orang
memilih
banyak
lebih
ini
sementara
```

Penjelasan:

1. def cari_kata_unik(file_path):

Baris ini mendefinisikan fungsi dengan nama cari_kata_unik, yang memiliki satu parameter file_path. Parameter ini mewakili lokasi file teks yang akan dibaca oleh program.

2. try:

Menandakan bahwa baris-baris selanjutnya akan diperiksa untuk kemungkinan terjadi error. Jika ada error saat membuka atau membaca file, program akan berpindah ke bagian except.

3. with open(file_path, 'r') as file:

Membuka file teks dari lokasi yang diberikan oleh file_path. Mode 'r' berarti file dibuka untuk dibaca (read-only).
Kata kunci with digunakan agar file otomatis ditutup setelah selesai dibaca, tanpa perlu menuliskan file.close() secara manual.

4. artikel = file.read()

Membaca seluruh isi file dan menyimpannya ke dalam variabel artikel. Tipe data hasil dari read() adalah string.

5. artikel = artikel.lower()

Mengubah seluruh huruf dalam string artikel menjadi huruf kecil (lowercase).
Hal ini bertujuan agar kata yang sama tetapi menggunakan huruf besar dan kecil (misalnya Python dan python) dianggap sama.

6. daftar_kata = artikel.split()

Memisahkan isi string artikel menjadi kata-kata individual. Fungsi split() memisahkan kata berdasarkan spasi atau whitespace.
Hasilnya adalah list (daftar) dari kata-kata dalam teks.

7. kata_unik = set(daftar_kata)

Mengonversi daftar_kata menjadi set, yaitu struktur data yang hanya menyimpan nilai-nilai unik (tanpa duplikat).

8. print("Kata-kata unik yang ditemukan:")

Menampilkan judul ke layar sebelum daftar kata unik ditampilkan satu per satu.

9. for kata in kata_unik:

Melakukan perulangan (loop) untuk setiap elemen (kata) di dalam kata_unik.

10. print(kata)

Dalam setiap iterasi, mencetak satu kata dari kata_unik ke layar. Karena set tidak terurut, urutan kata yang dicetak bisa berbeda setiap kali program dijalankan.

11. except FileNotFoundError:

Jika file yang diberikan tidak ditemukan pada path yang dimasukkan, maka baris ini akan dijalankan.

12. print(f"File {file_path} tidak ditemukan.")

Menampilkan pesan error yang menjelaskan bahwa file tidak ditemukan.

13. except Exception as e:

Menangkap error lain yang tidak spesifik, misalnya error membaca file yang rusak atau encoding error.

14. print(f"Terjadi kesalahan: {e}")

Menampilkan pesan error yang terjadi, agar pengguna tahu apa masalahnya.

15. file_path = input("Masukkan path file teks: ")

Meminta pengguna untuk memasukkan lokasi atau path file teks yang ingin diproses.

16. cari_kata_unik(file_path)

Memanggil fungsi cari_kata_unik dengan path yang telah dimasukkan pengguna agar seluruh proses di atas berjalan.